Glimmer DSL for Web

Ruby Web Frontend Framework
Ruby Web フロントエンド フレームワーク

Fukuoka Prefecture Future IT Initiative 2025

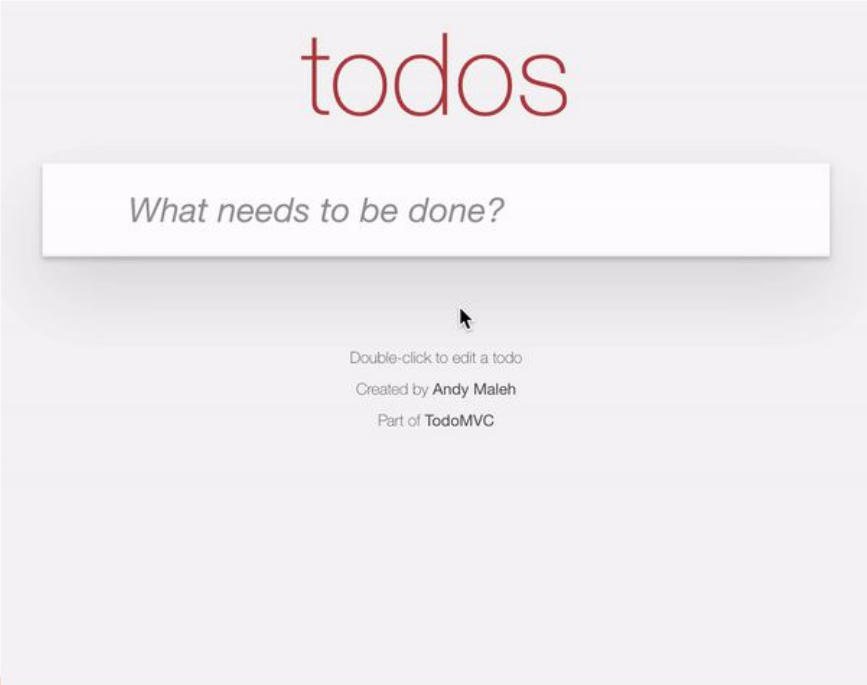
福岡県未来ITイニシアティブ2025

Andy Maleh (Montreal, Quebec, Canada)

Description of the Features of the Technology
技術の特徴の説明

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web is an open-source Opal Ruby Frontend Framework that can be used to build Frontend features in Rails/non-Rails web applications using Ruby in the Frontend instead of JavaScript.



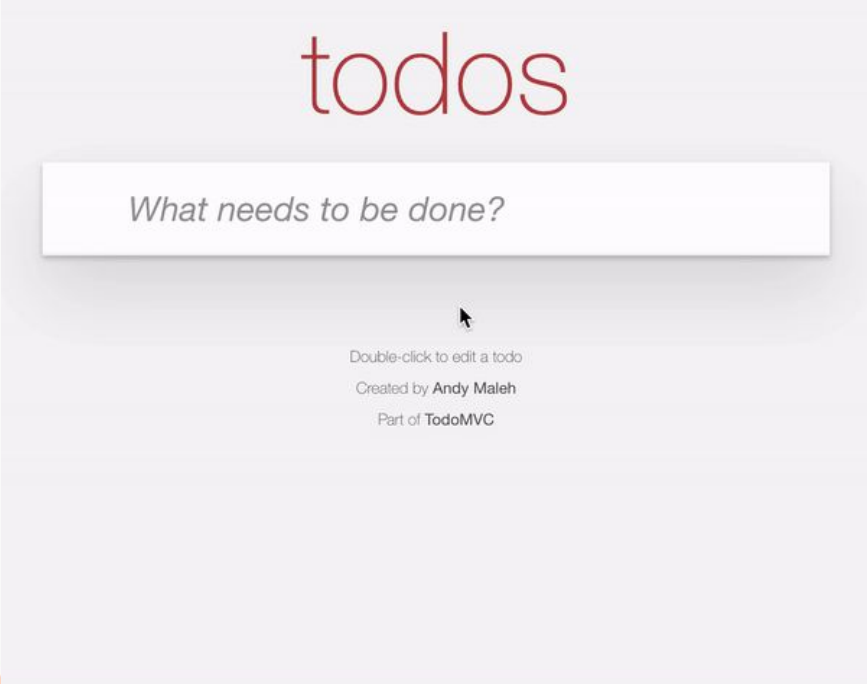
Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、

オープンソースの Opal Ruby フロントエンド フレームワークです。

JavaScript の代わりにフロントエンドで Ruby を使用し

Railsおよび非 Rails Web アプリケーションのフロントエンド機能を構築することができます。



TodoMVC Example
例

View
ビュー

Model or Presenter
モデルまたはプレゼンター

HTML

CSS

```
10  class TodoMvc
11    include Glimmer::Web::Component
12
13    before_render do
14      @presenter = TodoPresenter.new
15    end
16
17    after_render do
18      @presenter.setup_filter_routes
19    end
20
21    markup {
22      div {
23        section(class: 'todoapp') {
24          new_todo_form(presenter: @presenter)
25          todo_list(presenter: @presenter)
26          todo_filters(presenter: @presenter)
27        }
28        todo_mvc_footer
29
30        on_remove do
31          @presenter.unsetup_filter_routes
32        end
33      }
34    }
35
36    style {
37      r('body, button, html') {
38        margin 0
39        padding 0
40      }
41
42      r('button') {
43        _webkit_font_smoothing :antialiased
44        _webkit_appearance :none
```

todos

What needs to be done?

Shopping

Cleaning

Fixing The Deck

3 items left All Active Completed

Double-click to edit a todo
Created by Andy Maleh
Part of TodoMVC

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports building visual structure with a Ruby HTML DSL (Domain Specific Language), including SVG.

Ruby HTML DSL

```ruby
html {
  head {
    title('Contact Manager')
  }
  body {
    form(action: '/search_contacts') {
      div {
        label('Name: ', for: 'name-field')
        input(type: 'text', id: 'name-field')
      }
      div {
        input(type: 'submit', value: 'Search')
      }
    }
  }
}
```

HTML

```html
<html>
  <head>
    <title>Contact Manager</title>
  </head>
  <body>
    <form action="/search_contacts">
      <div>
        <label for="name-field" Name:   />
        <input type="text" id="name-field"  />
      </div>
      <div>
        <input type="submit" value="Search"  />
      </div>
    </form>
  </body>
</html>
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、SVG を含む Ruby HTML DSL (ドメイン固有言語) を使用して視覚的な構造を構築することをサポートします。

Ruby HTML DSL

```
html {
  head {
    title('Contact Manager')
  }
  body {
    form(action: '/search_contacts') {
      div {
        label('Name: ', for: 'name-field')
        input(type: 'text', id: 'name-field')
      }
      div {
        input(type: 'submit', value: 'Search')
      }
    }
  }
}
```

HTML

```
<html>
  <head>
    <title>Contact Manager</title>
  </head>
  <body>
    <form action="/search_contacts">
      <div>
        <label for="name-field" Name:   />
        <input type="text" id="name-field"  />
      </div>
      <div>
        <input type="submit" value="Search"  />
      </div>
    </form>
  </body>
</html>
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports styling elements with a Ruby CSS DSL (Domain Specific Language).

Ruby CSS DSL

```
body {
  font_size 1.1.em
  background :white
  width 90.%
  height 100.%
}
media('screen and (max-width: 600px)') {
  rule('body#app h1.title') {
    font_size 16
    font_family '"Times New Roman", serif'
  }
}
```

CSS

```
body {
  font-size: 1.1em;
  background: white;
  width: 90%;
  height: 100%;
}

@media screen and (max-width: 600px) {
  body#app h1.title{
    font-size: 16px;
    font-family: "Times New Roman", serif
  }
}
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、Ruby CSS DSL (ドメイン固有言語) を使用した要素のスタイル設定をサポートします。

Ruby CSS DSL

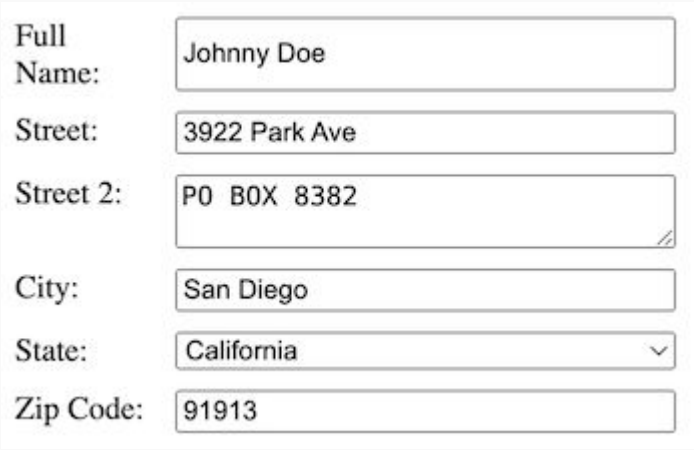
```
body {
  font_size 1.1.em
  background :white
  width 90.%
  height 100.%
}
media('screen and (max-width: 600px)') {
  rule('body#app h1.title') {
    font_size 16
    font_family '"Times New Roman", serif'
  }
}
```

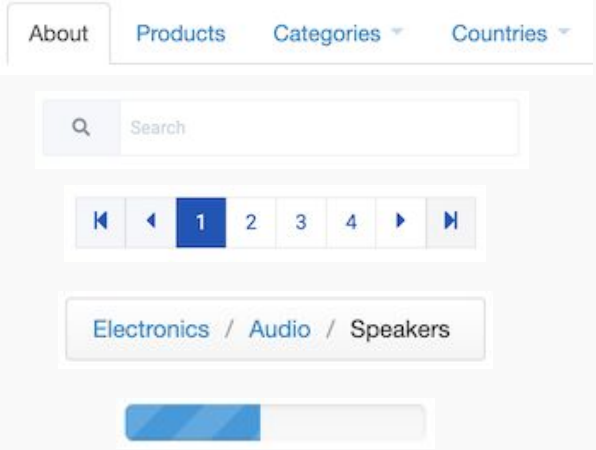
CSS

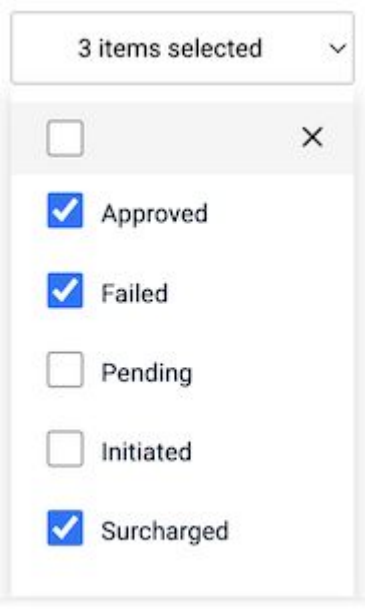
```
body {
  font-size: 1.1em;
  background: white;
  width: 90%;
  height: 100%;
}

@media screen and (max-width: 600px) {
  body#app h1.title{
    font-size: 16px;
    font-family: "Times New Roman", serif
  }
}
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports building custom visual components with Glimmer Web Components

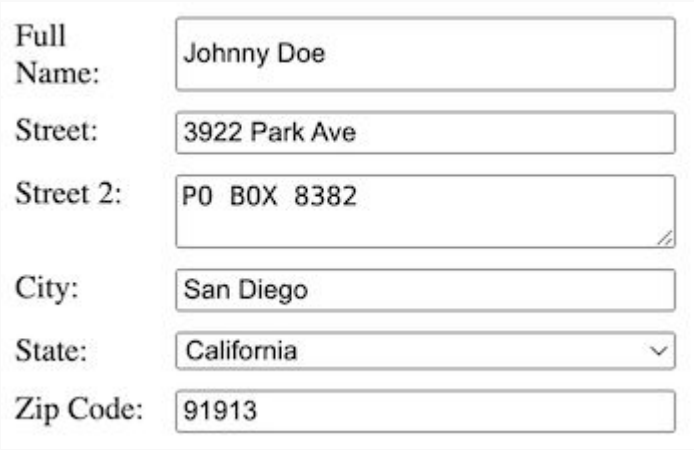


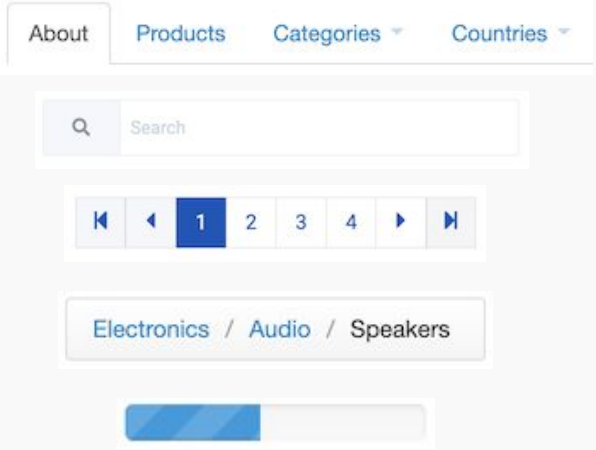


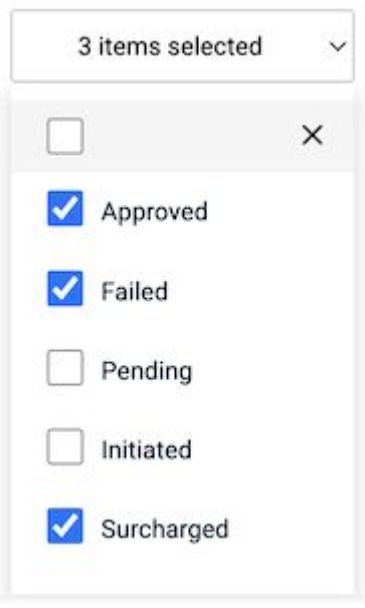


Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、Glimmer Web Componentを使用しカスタムビジュアルコンポーネントの構築をサポートしています。







Description of the Features of the Technology
技術の特徴の説明

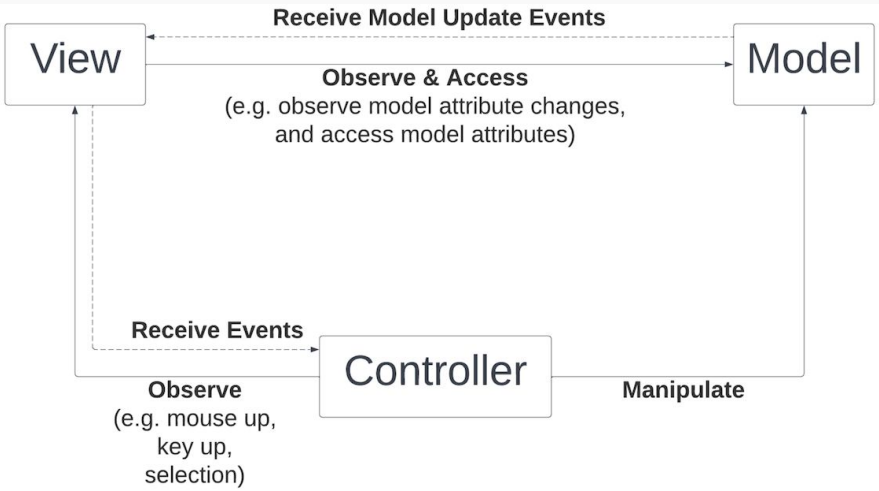
Glimmer DSL for Web supports the MVC (Model-View-Controller) Architectural Pattern using the Observer Design Pattern.

```
observe(@model, :attribute_name) do |value|
  $$.alert("Attribute value changed to: #{value}")
end
```

Model Observer
モデルオブザーバー

```
button('Click') {
  onclick do
    $$.alert('Button clicked!')
  end
}
```

View Observer
オブザーバーを表示



Description of the Features of the Technology
技術の特徴の説明

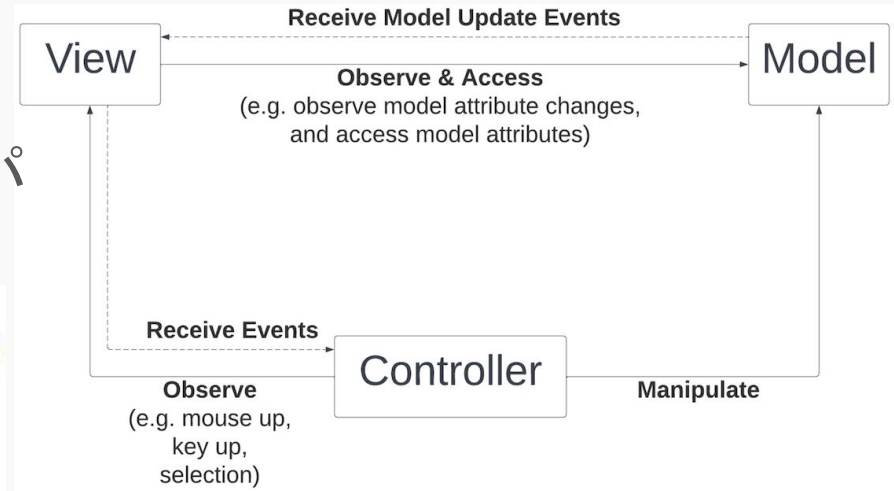
Glimmer DSL for Web は、Observer デザイン パターンを使用しMVC (モデル - ビュー - コントローラー) アーキテクチャ パターンをサポートしています。

```
observe(@model, :attribute_name) do |value|
  $$.alert("Attribute value changed to: #{value}")
end
```

Model Observer
モデルオブザーバー

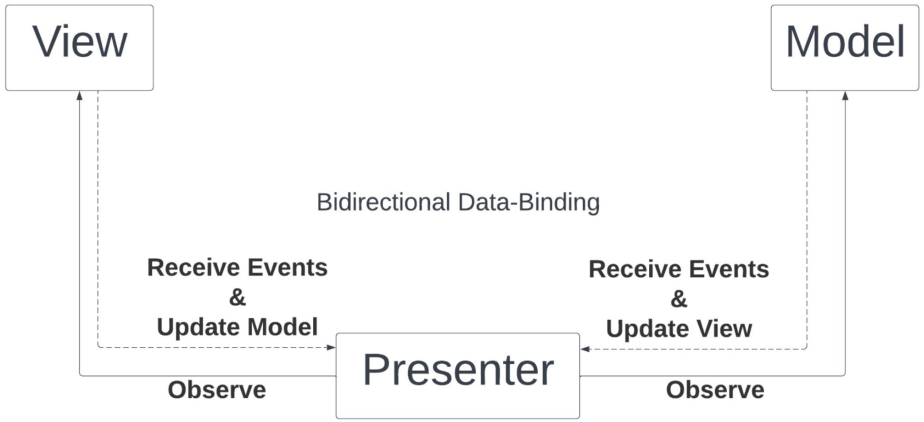
```
button('Click') {
  onclick do
    $$.alert('Button clicked!')
  end
}
```

View Observer
オブザーバーを表示



Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports the MVP (Model-View-Presenter) Architectural Pattern using Data-Binding.



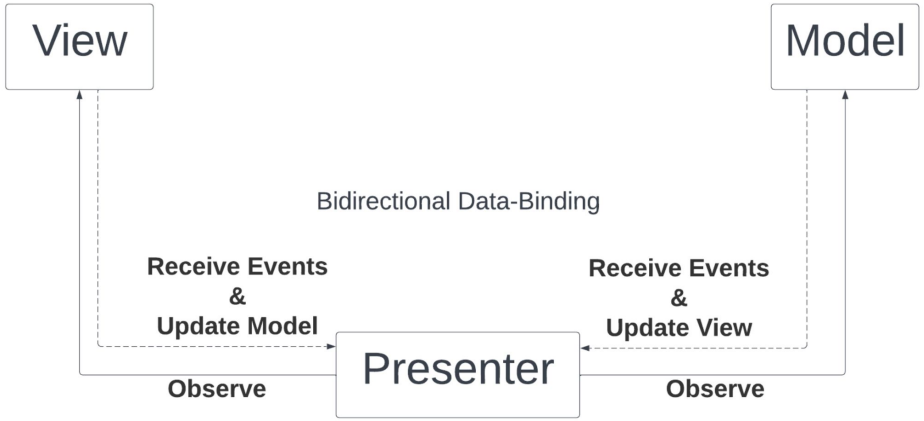

```
input(type: 'email') {
  value <=> [user, :email]
}

div {
  inner_text <= [address, :summary, computed_by: [:street, :city, :state, :zip]]
}
```

Bidirectional Data-Binding 双方向データバインディング

Unidirectional Data-Binding 一方向データバインディング

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、データ バインディングを使用し MVP (モデル - ビュー - プレゼンター) アーキテクチャパターンをサポートしています。




```
input(type: 'email') {
  value <=> [user, :email]
}

div {
  inner_text <= [address, :summary, computed_by: [:street, :city, :state, :zip]]
}
```

Bidirectional Data-Binding 双方向データバインディング

Unidirectional Data-Binding 一方向データバインディング

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports converting legacy HTML code into Glimmer DSL code with the html_to_glimmer command.

```
<html>
  <head>
    <title>Contact Manager</title>
  </head>
  <body>
    <form action="/search_contacts">
      <div>
        <label for="name-field" Name:   />
        <input type="text" id="name-field"  />
      </div>
      <div>
        <input type="submit" value="Search"  />
      </div>
    </form>
  </body>
</html>
```

→

```
html {
  head {
    title('Contact Manager')
  }
  body {
    form(action: '/search_contacts') {
      div {
        label('Name: ', for: 'name-field')
        input(type: 'text', id: 'name-field')
      }
      div {
        input(type: 'submit', value: 'Search')
      }
    }
  }
}
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、html_to_glimmer コマンドを使用し従来のHTML コードを Glimmer DSL コードに変換することをサポートしています。

```
<html>
  <head>
    <title>Contact Manager</title>
  </head>
  <body>
    <form action="/search_contacts">
      <div>
        <label for="name-field" Name:   />
        <input type="text" id="name-field"  />
      </div>
      <div>
        <input type="submit" value="Search"  />
      </div>
    </form>
  </body>
</html>
```

→

```
html {
  head {
    title('Contact Manager')
  }
  body {
    form(action: '/search_contacts') {
      div {
        label('Name: ', for: 'name-field')
        input(type: 'text', id: 'name-field')
      }
      div {
        input(type: 'submit', value: 'Search')
      }
    }
  }
}
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports converting legacy CSS code into Glimmer DSL code with the css_to_glimmer command.

```
body {
  font-size: 1.1em;
  background: white;
  width: 90%;
  height: 100%;
}

@media screen and (max-width: 600px) {
  body#app h1.title{
    font-size: 16px;
    font-family: "Times New Roman", serif
  }
}
```

→

```
body {
  font_size 1.1.em
  background :white
  width 90.%
  height 100.%
}
media('screen and (max-width: 600px)') {
  rule('body#app h1.title') {
    font_size 16
    font_family '"Times New Roman", serif'
  }
}
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、css_to_glimmer コマンドを使用し従来の CSS コードを Glimmer DSL コードに変換することをサポートしています。

```
body {
  font-size: 1.1em;
  background: white;
  width: 90%;
  height: 100%;
}

@media screen and (max-width: 600px) {
  body#app h1.title{
    font-size: 16px;
    font-family: "Times New Roman", serif
  }
}
```

→

```
body {
  font_size 1.1.em
  background :white
  width 90.%
  height 100.%
}
media('screen and (max-width: 600px)') {
  rule('body#app h1.title') {
    font_size 16
    font_family '"Times New Roman", serif'
  }
}
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports making HTTP REST API web requests with the HTTP class.

```
HTTP.get(url, options) { |response| … }
HTTP.post(url, options) { |response| … }
HTTP.put(url, options) { |response| … }
HTTP.delete(url, options) { |response| … }
HTTP.patch(url, options) { |response| … }
HTTP.head(url, options) { |response| … }
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、HTTP クラスを使用しHTTP REST API Webリクエストの作成をサポートしています。

```
HTTP.get(url, options) { |response| … }
HTTP.post(url, options) { |response| … }
HTTP.put(url, options) { |response| … }
HTTP.delete(url, options) { |response| … }
HTTP.patch(url, options) { |response| … }
HTTP.head(url, options) { |response| … }
```

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web supports integration with any JavaScript libraries using the Opal Ruby require statement or HTML script tag with src attribute.

```
require 'vendor/jquery'
```

Opal Ruby require statement
オパールルビーの要求ステートメント

```
script(src: 'https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/highlight.min.js')
```

HTML script tag
HTML スクリプト タグ

Description of the Features of the Technology
技術の特徴の説明

Glimmer DSL for Web は、Opal Ruby の require ステートメントまたは src 属性を持つ HTML スクリプト タグを使用し任意の JavaScript ライブラリとの統合をサポートしています。

```
require 'vendor/jquery'
```

Opal Ruby require statement
オパールルビーの要求ステートメント

```
script(src: 'https://cdnjs.cloudflare.com/ajax/libs/highlight.js/11.9.0/highlight.min.js')
```

HTML script tag
HTML スクリプト タグ

How Advanced and Creative the Product Is
製品の先進性と創造性

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web enables programming logic, structure, and style in one language, Ruby:

- Logic is written with Opal (Ruby code)
- Structure is written with Glimmer HTML DSL (Ruby code)
- Style is written with Glimmer CSS DSL (Ruby code)

```
table(class: 'samples') {
  tbody {
    @presenter.samples.each do |sample|
      tr {
        td(sample.name)
        class_name <= [@presenter, :selected_sample,
                       on_read: -> (val) { val == sample ? 'selected' : '' }
                      ]
        onclick do |event|
          event.prevent_default
          @presenter.selected_sample = sample
        end
      }
    end
  }
  style {
    r('table.samples tr td') {
      border '1px solid transparent'
      padding @presenter.table_cell_padding
      cursor :default
    }
  }
}
```

Structure 構造

Logic 論理

Structure 構造

Logic 論理

Style スタイル

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web では、プログラミング ロジック、構造、スタイルを 1 つの言語 (Ruby) で記述できます。

- ロジックは Opal (Ruby コード) で記述されます
- 構造は Glimmer HTML DSL (Ruby コード) で記述されます
- スタイルは Glimmer CSS DSL (Ruby コード) で記述されます

```
table(class: 'samples') {
  tbody {
    @presenter.samples.each do |sample|
      tr {
        td(sample.name)
        class_name <= [@presenter, :selected_sample,
                       on_read: -> (val) { val == sample ? 'selected' : '' }
                      ]
        onclick do |event|
          event.prevent_default
          @presenter.selected_sample = sample
        end
      }
    end
  }
  style {
    r('table.samples tr td') {
      border '1px solid transparent'
      padding @presenter.table_cell_padding
      cursor :default
    }
  }
}
```

Structure 構造

Logic 論理

Structure 構造

Logic 論理

Style スタイル

How Advanced and Creative the Product Is
製品の先進性と創造性

ERB ❌

```
<table>
  <tbody>
    <% @books.each do |book| %>
      <tr>
        <td><%= book.title %></td>
        <td><%= book.content %></td>
        <td><%= book_link(book) %></td>
        <% if book.editable? %>
          <td><%= edit_book_link(book) %></td>
        <% end %>
      </tr>
    <% end %>
  </tbody>
</table>
```

Glimmer ✅

```
table {
  tbody {
    @books.each do |book|
      tr {
        td(book.title)
        td(book.content)
        td { book_link(book) }
        if book.editable?
          td { edit_book_link(book) }
        end
      }
    end
  }
}
```

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web enables dividing and conquering complexity by building visual features as Glimmer Web Components with support for Component Slots, Component Custom Event Listeners, and Component Embedding in Rails ERB files.

Glimmer DSL for Web は、コンポーネント スロット、コンポーネント カスタム イベント リスナー、および Rails ERB ファイルへのコンポーネント埋め込みをサポートする Glimmer Web コンポーネントとして視覚的な機能を構築することで、複雑さを分割して克服することを可能にします。

```ruby
class AddressForm
  include Glimmer::Web::Component

  option :address

  markup {
    div {
      div(style: 'display: grid; grid-auto-columns: 80px 260px;') {
        label('Full Name: ', for: 'full-name-field')
        input(id: 'full-name-field') {
          value <=> [address, :full_name]
        }

        label('Street: ', for: 'street-field')
        input(id: 'street-field') {
          value <=> [address, :street]
        }

        label('Street 2: ', for: 'street2-field')
        textarea(id: 'street2-field') {
          value <=> [address, :street2]
        }

        label('City: ', for: 'city-field')
        input(id: 'city-field') {
          value <=> [address, :city]
        }

        label('State: ', for: 'state-field')
        select(id: 'state-field') {
          Address::STATES.each do |state_code, state|
            option(value: state_code) { state }
          end

          value <=> [address, :state_code]
        }
        ...
```

```ruby
class AddressForm
  include Glimmer::Web::Component

  option :address

  markup {
    div {
      div(style: 'display: grid; grid-auto-columns: 80px 260px;') {
        label('Full Name: ', for: 'full-name-field')
        input(id: 'full-name-field') {
          value <=> [address, :full_name]
        }

        label('Street: ', for: 'street-field')
        input(id: 'street-field') {
          value <=> [address, :street]
        }

        label('Street 2: ', for: 'street2-field')
        textarea(id: 'street2-field') {
          value <=> [address, :street2]
        }

        label('City: ', for: 'city-field')
        input(id: 'city-field') {
          value <=> [address, :city]
        }

        label('State: ', for: 'state-field')
        select(id: 'state-field') {
          Address::STATES.each do |state_code, state|
            option(value: state_code) { state }
          end

          value <=> [address, :state_code]
        }
        ...
```

```ruby
div {
  h1('Shipping Address')

  address_form(address: @shipping_address)

  h1('Billing Address')

  address_form(address: @billing_address)
}
```

Shipping Address

Full Name: Johnny Doe

Street: 3922 Park Ave

Street 2: PO BOX 8382

City: San Diego

State: California

Zip Code: 91913

Billing Address

Full Name: John C Doe

Street: 123 Main St

Street 2: Apartment 3C

City: San Diego

State: California

Zip Code: 91911

```
div {
  h1('Shipping Address')

  address_form(address: @shipping_address)

  h1('Billing Address')

  address_form(address: @billing_address)
}
```

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web supports multiple types of Data-Binding:

- Element Property Data-Binding (Bidirectional & Unidirectional)
- Element Content Data-Binding
- CSS Inline Style Data-Binding
- CSS Class Inclusion Data-Binding

Glimmer DSL for Web は、複数のタイプのデータ バインディングをサポートしています:

- 要素プロパティ データ バインディング (双方向および一方向)
- 要素コンテンツ データ バインディング
- CSS インライン スタイル データ バインディング
- CSS クラス インクルード データ バインディング

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web supports advanced Data-Binding options:

- Converters
- Hooks
- Computed Data-Binding
- Nested Data-Binding
- Indexed Data-Binding
- Keyed Data-Binding

Glimmer DSL for Web は、高度なデータ バインディング オプションをサポートしています:

- コンバーター
- フック
- 計算されたデータ バインディング
- ネストされたデータ バインディング
- インデックス付きデータ バインディング
- キー付きデータ バインディング

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web Data-Binding/Observer observes Model attributes non-intrusively without requiring them to inherit from any class/module by automatically enhancing them as observables through Ruby meta-programming. This keeps Models pure of View details, which simplifies code and improves maintainability and productivity.

Web データバインディング/オブザーバー用の Glimmer DSL は、Ruby メタプログラミングを通じてモデル属性をオブザーバブルとして自動的に拡張することで、クラス/モジュールから継承する必要なく、モデル属性を非侵入的に監視します。これにより、モデルはビューの詳細から純粋な状態になり、コードが簡素化され、保守性と生産性が向上します。

How Advanced and Creative the Product Is
製品の先進性と創造性

Glimmer DSL for Web can render 1000 elements in ~0.055 of a second on a MacBook Pro 14" 2023 (M2 Chip). That is more than fast enough for most business applications as it is way below the 0.500s-1.000s range recommended for websites.

Glimmer DSL for Web は、MacBook Pro 14 インチ 2023 (M2 チップ) で 1000 要素を約 0.055 秒でレンダリングできます。これは、Web サイトに推奨される 0.500 秒から 1.000 秒の範囲をはるかに下回るため、ほとんどのビジネス アプリケーションには十分すぎる速度です。

The impact of the product on society
製品が社会に与える影響

The impact of the product on society
製品が社会に与える影響

With Glimmer DSL for Web, programmers do not waste time unnecessarily splitting their resources across multiple languages as they focus on developing the Frontend and Backend with one very advanced and expressive language, Ruby, so they end up with less code to maintain overall, which improves productivity and maintainability.

Glimmer DSL for Web を使用すると、プログラマーは、非常に高度で表現力豊かな 1 つの言語である Ruby を使用してフロントエンドとバックエンドの開発に集中できるため、リソースを複数の言語に不必要に分割して時間を無駄にすることがなくなり、全体的に保守するコードが少なくなり、生産性と保守性が向上します。

The impact of the product on society
製品が社会に与える影響

With Glimmer DSL for Web, programmers become happy by using Opal Ruby due to avoiding JavaScript badly engineered, over-engineered, and premature-optimization-obsessed libraries (e.g. React, Angular, Vue, Ember, and Svelte) and not fighting JavaScript build issues (e.g. webpack) anymore.

Glimmer DSL for Web を使用すると、JavaScript の不適切な設計、過剰な設計、および早期最適化にこだわったライブラリ (React、Angular、Vue、Ember、Svelte など) を回避し、JavaScript ビルドの問題 (webpack など) と格闘する必要がなくなるため、プログラマーは Opal Ruby を使用することで満足できるようになります。

The impact of the product on society
製品が社会に与える影響

With Glimmer DSL for Web, customers are provided with a better user experience because programmers have the ability to reuse Backend Ruby logic in the Frontend as is, so that removes the need to make a lot of HTTP web requests, which speeds up the response time of user actions in the Frontend.

Glimmer DSL for Web を使用すると、プログラマーはバックエンドの Ruby ロジックをそのままフロントエンドで再利用できるため、多くの HTTP Web リクエストを行う必要がなくなり、フロントエンドでのユーザーアクションの応答時間が短縮され、顧客に優れたユーザーエクスペリエンスが提供されます。

The impact of the product on society
製品が社会に与える影響

With Glimmer DSL for Web, web application frontends are completed in about half the time and cost for customers, which gives businesses a very big competitive advantage as they can finish 12 months of Frontend Development work in about 6 months using Ruby in the Frontend.

Glimmer DSL for Web を使用すると、Web アプリケーションのフロントエンドが顧客の約半分の時間とコストで完成します。これにより、フロントエンドで Ruby を使用して 12 か月かかるフロントエンド開発作業を約 6 か月で完了できるため、企業は非常に大きな競争上の優位性を獲得できます。

The impact of the product on society
製品が社会に与える影響

With Glimmer DSL for Web, companies cut their hiring budget in half as Backend Ruby Developers do Frontend development in Ruby, so they become Full-Stack Developers who produce better products for clients.

Glimmer DSL for Web を使用すると、バックエンド Ruby 開発者が Ruby でフロントエンド開発を行うため、企業は採用予算を半分に削減し、クライアントのためにより優れた製品を生み出すフルスタック開発者になることができます。

Glimmer DSL for Web Q & A
Glimmer DSL for Web 質疑応答

- GitHub: https://github.com/AndyObtiva/glimmer-dsl-web
- RubyGem: https://rubygems.org/gems/glimmer-dsl-web
- Andy Maleh LinkedIn: https://www.linkedin.com/in/andymaleh/
- Presentation Slides: https://docs.google.com/presentation/d/e/2PACX-1vQ8_DEsug8pIaHTKtVM1Iye0I22gU-tatPT1B5WEwp6uRXkfmlQ9SSmdmCTHvXuPDb749wvPzBUmqRz/pub